


```
}  
  
}
```

Ejercicio 3

Se pide implementar la clase `ConcatIterator` la cual, recibe dos iteradores y genera la concatenación de ambos.

Implementar todo lo necesario para que, con el siguiente programa de prueba

```
import java.util.Iterator;  
import java.util.List;  
  
public class ConcatIteratorTester {  
  
    public static void main(String[] args) {  
        List<String> list1 = List.of("a", "b", "c", "d");  
        List<String> list2 = List.of("1", "2", "3", "4");  
        List<String> list3 = List.of("alpha", "beta");  
        Iterator<String> iterator = new ConcatIterator<>(  
            new ConcatIterator<>(list1.iterator(), list2.iterator()),  
            list3.iterator());  
        while(iterator.hasNext()) {  
            System.out.print(iterator.next());  
        }  
    }  
}
```

se obtenga la siguiente salida

```
abcd1234alphabeta
```

Ejercicio 4

Se está implementando un sistema para rankear libros. Luego, dicha información se puede recuperar para generar un listado de los mismos ordenados por ranking. Además, los libros se pueden asociar a un género. También se puede generar un listado por género, también ordenado. Si un libro no tiene género, se lo puede agregar luego.

Notar que las invocaciones al método de instancia `add` están encadenadas. ¿Qué tipo de dato debe retornar `add` para poder invocar otro método de instancia de `Ranker`?

Implementar todo lo necesario para que, con el siguiente programa de prueba

```
public class RankerTester {  
  
    public static void main(String[] args) {  
        Ranker ranker = new Ranker();  
  
        Book hp7 = new Book("Harry Potter and the Deadly Hallows", "JK Rowling");  
        Book t2t = new Book("The Two Towers", "JRR Tolkien");  
        Book theHobbit = new Book("The Hobbit", "JRR Tolkien");  
        Book studyInScarlet = new Book("A Study in Scarlet", "Arthur Conan Doyle");
```

```

Book hamlet = new Book("Hamlet", "William Shakespeare");
Book prejudice = new Book("Pride and Prejudice", "Jane Austen");
Book eragon = new Book("Eragon", "Christopher Paolini");

ranker.add(Genre.FANTASY, hp7).add(Genre.FANTASY, theHobbit)
      .add(Genre.FANTASY, t2t).add(Genre.FICTION, studyInScarlet)
      .add(Genre.DRAMA, hamlet).add(Genre.DRAMA, prejudice);

ranker.rateUp(hp7);
ranker.rateUp(hp7);
ranker.rateUp(hp7);
ranker.rateUp(theHobbit);
ranker.rateUp(theHobbit);
ranker.rateUp(hamlet);
ranker.rateUp(new Book("Eragon", "Christopher Paolini"));
ranker.rateUp(eration);

ranker.printRanking(Genre.FANTASY);
System.out.println("-----");
ranker.printRanking(Genre.DRAMA);
System.out.println("-----");
ranker.printRanking(Genre.CHILDREN);
System.out.println("-----");
ranker.printRanking();
}
}

```

se obtenga la siguiente salida:

```

Ranking of Fantasy
Harry Potter and the Deadly Hallows : 3
The Hobbit : 2
The Two Towers : 0
-----
Ranking of Drama
Hamlet : 1
Pride and Prejudice : 0
-----
Ranking of Children
-----
General Ranking
Harry Potter and the Deadly Hallows : 3
Eragon : 2
The Hobbit : 2
Hamlet : 1
A Study in Scarlet : 0
Pride and Prejudice : 0
The Two Towers : 0

```

Ejercicio 5

Se cuenta con la interfaz `IterableBag` que extiende a la interfaz `Bag` del *Ejercicio 8* del *TP N°6* y permite iterar por los elementos de la bolsa de dos formas:

```

public interface IterableBag<E> extends Bag<E> {

```

```

/**
 * Para iterar, en orden descendente, por todos los elementos que hay en la bolsa.
 */
Iterable<E> elements();

/**
 * Para iterar, en orden descendente, por todos los elementos distintos
 * que hay en la bolsa.
 */
Iterable<E> elementsDistinct();
}

```

Realizar el diagrama de clases. Implementar todo lo necesario para que, con el siguiente programa:

```

public class IterableBagTester {

    public static void main(String[] args) {
        IterableBag<String> stringBag = new IterableBagImpl<>();
        System.out.println(stringBag.contains("hola"));
        stringBag.add("hola");
        System.out.println(stringBag.contains("hola"));
        stringBag.add("que");
        stringBag.add("que");
        System.out.println(stringBag.count("que"));
        stringBag.add("tal");
        System.out.println(stringBag.count("que"));
        System.out.println(stringBag.sizeDistinct());
        for(String elem : stringBag.elementsDistinct()) {
            System.out.println(elem);
        }
        System.out.println(stringBag.size());
        for(String elem : stringBag.elements()) {
            System.out.println(elem);
        }
        stringBag.remove("tal");
        System.out.println(stringBag.sizeDistinct());
        System.out.println("-----");
        IterableBag<Integer> integerBag = new IterableBagImpl<>();
        integerBag.remove(2);
    }
}

```

se obtenga la siguiente salida:

```

false
true
2
2
3
tal
que
hola

```

```
4
tal
que
que
hola
2
-----
Exception in thread "main" java.util.NoSuchElementException
```

¿Es necesario establecer alguna restricción sobre el tipo genérico E?

Ejercicio 6

Un multimapa es un mapa que admite más de un valor para una misma clave. Se cuenta con la siguiente interfaz `MultiMap`.

```
public interface MultiMap<K, V>{

    /**
     * Agrega un par key,value al multimapa si el par no existe.
     */
    void put(K key, V value);

    /**
     * Cantidad de valores del multimapa.
     */
    int size();

    /**
     * Cantidad de valores del multimapa para la clave key.
     */
    int size(K key);

    /**
     * Elimina la clave del multimapa (con todos sus valores) si existe.
     */
    void remove(K key);

    /**
     * Elimina el valor value de la clave key si existe.
     */
    void remove(K key, V value);

    /**
     * Colección ordenada descendentemente de valores de clave key.
     */
    Iterable<V> get(K key);

}
```

Implementar todo lo necesario (pudiéndose modificar la interfaz) para que, con el siguiente programa,

```
public class MultiMapTester {
```

```

public static void main(String[] args) {
    MultiMap<String,Integer> m = new MultiMapImpl<>();
    m.put("hola", 4);
    m.put("hola", 3);
    m.put("hola", 2);
    m.put("chau", 4);
    m.put("chau", 5);
    m.put("adios", 6);
    System.out.println(m.size());
    System.out.println("#####");
    for(Integer i : m.get("hola")) {
        System.out.println(i);
    }
    System.out.println("#####");
    m.remove("adios");
    m.remove("hola", 2);
    for(Integer i : m.get("hola")) {
        System.out.println(i);
    }
    System.out.println("#####");
    System.out.println(m.get("adios"));
    System.out.println("#####");
    System.out.println(m.size());
}
}

```

se obtenga la siguiente salida

```

6
#####
4
3
2
#####
4
3
#####
null
#####
4

```

Ejercicio 7

Se cuenta con una interfaz `SortedMap` (una propia, no la del paquete `java.util`) que modela a un mapa ordenado por las claves y permite obtener la primera clave y el valor asociado a la primera clave con los siguientes métodos:

```

public interface SortedMap<K,V> {

    K firstKey();

    V firstValue();

}

```

Implementar todo lo necesario (pudiéndose modificar la interfaz) para que con el siguiente programa de prueba

```
public class SortedMapTester {  
  
    public static void main(String[] args) {  
        SortedMap<MyComparableClass, String> sm = new SortedMapImpl<>();  
        MyComparableClass obj1 = new MyComparableClass("A");  
        MyComparableClass obj2 = new MyComparableClass("B");  
        MyComparableClass obj3 = new MyComparableClass("C");  
        MyComparableClass obj4 = new MyComparableClass("D");  
        obj1.sortableIdentifier = "Key 1";  
        obj2.sortableIdentifier = "Key 2";  
        obj3.sortableIdentifier = "Key 3";  
        obj4.sortableIdentifier = "Key 4";  
        sm.put(obj1, "Value 1");  
        sm.put(obj2, "Value 2");  
        sm.put(obj3, "Value 3");  
        sm.put(obj4, "Value 4");  
        System.out.println("-----");  
        for(Map.Entry<MyComparableClass, String> each : sm) {  
            System.out.println(each.getKey());  
        }  
        System.out.println("-----");  
        System.out.println(sm.firstKey());  
        System.out.println(sm.firstValue());  
  
        sm.remove(obj1);  
        sm.remove(obj2);  
        sm.remove(obj4);  
        sm.remove(obj3);  
        obj1.sortableIdentifier = "Key 4";  
        obj2.sortableIdentifier = "Key 3";  
        obj3.sortableIdentifier = "Key 2";  
        obj4.sortableIdentifier = "Key 1";  
        sm.put(obj1, "Value 1");  
        sm.put(obj2, "Value 2");  
        sm.put(obj3, "Value 3");  
        sm.put(obj4, "Value 4");  
        System.out.println("-----");  
        for(Map.Entry<MyComparableClass, String> each : sm) {  
            System.out.println(each.getKey());  
        }  
        System.out.println("-----");  
        System.out.println(sm.firstKey());  
        System.out.println(sm.firstValue());  
    }  
}
```

se obtenga la siguiente salida:

```
-----  
A - (Key 1)  
B - (Key 2)  
C - (Key 3)
```

```

D - (Key 4)
-----
A - (Key 1)
Value 1
-----
D - (Key 1)
C - (Key 2)
B - (Key 3)
A - (Key 4)
-----
D - (Key 1)
Value 4

```

Analizando el programa de prueba responder:

- La interfaz `SortedMap` ¿debería extender de alguna otra interfaz? ¿Por qué?
- ¿Por qué en el programa de prueba se eliminan y se vuelven a agregar los datos una vez cambiado el `sortableIdentifier`? Justificar

Ejercicio 8

Retomando la implementación de la clase `BankAccount` del *Ejercicio 4* del *TP N°6* ahora se desea contar con un nuevo tipo de cuenta corriente para modelar a los clientes especiales. La misma ofrece descuentos de manera automática para las compras realizadas en ciertos locales. Los locales que participan de dichas promociones son obtenidos a partir del `ShopDiscountsProvider` mediante el método `getDiscount`. Dicho método devuelve el porcentaje de descuento a aplicar al importe a debitar.

A continuación se presenta la implementación del `ShopDiscountsProvider`:

```

import java.util.HashMap;
import java.util.Map;

public class ShopDiscountsProvider {

    private final Map<String, Double> discounts = new HashMap<>();

    public void addShop(String shop, double discount) {
        this.discounts.put(shop, discount);
    }

    public double getDiscount(String shop) {
        return discounts.getOrDefault(shop, 0.0);
    }

}

```

Realizar la implementación de la clase `PremiumAccount` que modele las cuentas para los clientes especiales. El constructor de `PremiumAccount` recibe el id de la cuenta, el descubierta de cuenta corriente y la instancia de `ShopDiscountProvider` para obtener los descuentos correspondientes.

El siguiente programa demuestra la manera de uso de `PremiumAccount`:

```

public class PremiumAccountTester {

```



```
public static void main(String[] args) {  
    BankAccount checkingAccount = new CheckingAccount(1234, 5000);  
    c1.deposit(200);  
    c1.extract(300);  
    checkingAccount.showMovements();  
  
    ShopDiscountsProvider shopDiscountsProvider = new ShopDiscountsProvider();  
    shopDiscountsProvider.addShop("Falabella", 0.1D);  
    shopDiscountsProvider.addShop("Nike", 0.15D);  
    shopDiscountsProvider.addShop("Garbarino", 0.3D);  
  
    PremiumAccount premiumAccount = new PremiumAccount(9999, 5000,  
shopDiscountsProvider);  
    premiumAccount.deposit(1000);  
    premiumAccount.extract(150, "Nike");  
    premiumAccount.extract(250, "Lacoste");  
    premiumAccount.extract(50, "Starbucks");  
    premiumAccount.extract(150, "Nike");  
    premiumAccount.showMovements();  
}  
}
```

Al ejecutar el programa se obtiene por consola:

```
Movements for Account 1234  
Deposit $200.0  
Extraction $300.0  
Movements for Account 9999  
Deposit $1000.0  
Extraction $127.5 for shop Nike  
Extraction $250.0 for shop Lacoste  
Extraction $50.0 for shop Starbucks  
Extraction $127.5 for shop Nike
```

Como se ve en el ejemplo, ahora es necesario guardar un registro de todos los movimientos realizados en las cuentas para consultarlo con el método `showMovements()`. ¿Cómo lo implementaría?

Ejercicio 9

Se desea implementar la clase **HelpDesk** que modela a un centro de atención a clientes de una empresa, donde se puede abrir **tickets de ayuda** con el método `openTicket`. Para abrir un ticket se debe indicar el tipo de cliente, el nombre del cliente y la pregunta.

Ya cuenta con el enum `ClientType` que modela a los tres tipos de cliente y no puede modificarse:

```
public enum ClientType {  
  
    GOVERNMENT, BUSINESS, RESIDENTIAL  
  
}
```

Se debe implementar un método `getNextTicket` que retorne el próximo ticket de ayuda a resolver a partir de un tipo de cliente y lo remueva. Se debe retornar el ticket mas antiguo con ese tipo de cliente. Si no hay tickets de ayuda por resolver para ese tipo de cliente lanza un error.

También se debe implementar un método `getNextTicket` que retorne el próximo ticket de ayuda a resolver considerando todos los tipos de clientes y lo remueva. Se debe retornar el ticket mas antiguo siempre considerando con mayor prioridad a los clientes gubernamentales primero, luego a los clientes empresariales y por último a los clientes residenciales.

Debe ofrecer también un método `hasTickets` que retorna true si existe al menos un ticket por resolver considerando todas las categorías de clientes.

Implementar todo lo necesario para que se obtenga la salida indicada en los comentarios:

```
public class HelpDeskTester {

    public static void main(String[] args) {
        HelpDesk helpDesk = new HelpDesk();

        // Abre un ticket para un cliente Business de nombre ITBA con la pregunta Question 1
        helpDesk.openTicket(ClientType.BUSINESS, "ITBA", "Question 1");

        helpDesk.openTicket(ClientType.RESIDENTIAL, "Foo", "Question 2")
            .openTicket(ClientType.BUSINESS, "ACME", "Question 3")
            .openTicket(ClientType.GOVERNMENT, "Arg", "Question 4")
            .openTicket(ClientType.RESIDENTIAL, "Bar", "Question 5")
            .openTicket(ClientType.RESIDENTIAL, "Foo", "Question 6");

        // Obtiene el próximo ticket a resolver para los clientes Business
        // en orden de apertura de los tickets y luego se lo elimina
        HelpDesk.HelpTicket ticket = helpDesk.getNextTicket(ClientType.BUSINESS);
        System.out.println(ticket); // HelpTicket for ITBA asking Question 1

        ticket = helpDesk.getNextTicket(ClientType.BUSINESS);
        System.out.println(ticket); // HelpTicket for ACME asking Question 3

        try {
            ticket = helpDesk.getNextTicket(ClientType.BUSINESS);
        } catch (Exception ex) {
            System.out.println(ex.getClass()); // class java.util.NoSuchElementException
        }

        // Obtiene el próximo ticket a resolver para los clientes de todas las categorías
        // tomando primero los clientes Government, luego Business y por último Residential
        // y desempata por orden de apertura de los tickets
        // y luego se lo elimina
        while(helpDesk.hasTickets()) {
            ticket = helpDesk.getNextTicket();
            System.out.println(ticket);
        }
        /**
         * HelpTicket for Arg asking Question 4
         * HelpTicket for Foo asking Question 2
         * HelpTicket for Bar asking Question 5
         * HelpTicket for Foo asking Question 6
         */
    }
}
```

Ejercicio 10

Modificar la implementación del **Ejercicio 9** para mantener el orden de resolución de tickets pedido pero ahora utilizando el siguiente `enum` (que no puede modificarse):

```
public enum ClientType {  
    RESIDENTIAL, BUSINESS, GOVERNMENT  
}
```